

MCP SERVER USAGE GUIDE

For Blockchain Integration Project

Hyperledger Fabric | CouchDB | PostgreSQL | Node.js API | Angular UI | Docker

Purpose

This document explains how to use MCP servers as an AI engineering assistant layer for the Blockchain Integration Project. It includes recommended MCP servers, project architecture, setup phases, security rules, dashboard use cases, and ready-to-use prompts.

Prepared for: Nicolas Salloum

Date: May 18, 2026

1. What is an MCP Server?

MCP means Model Context Protocol. An MCP server is a connector that allows an AI assistant or AI coding tool to safely access external systems such as files, databases, APIs, GitHub repositories, Docker containers, and browser automation tools.

For your Blockchain Integration Project, MCP can become the bridge between the AI assistant and your real technical environment.



2. Why MCP is Useful for Your Blockchain Integration Project

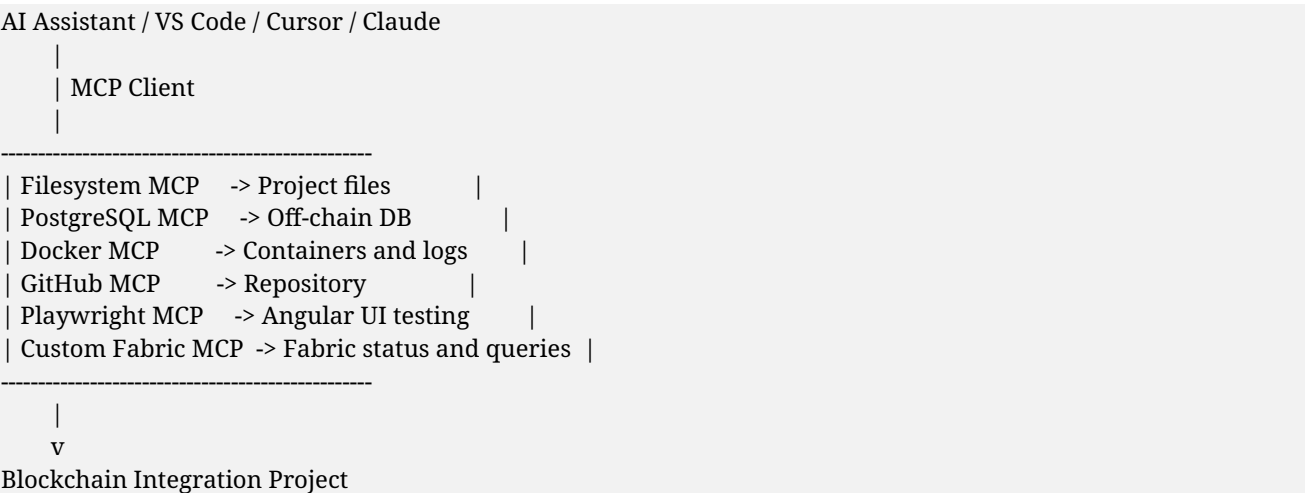
Project Area	How MCP Helps
Hyperledger Fabric files	Read network configuration, connection profiles, certificates path references, channel configuration, and chaincode deployment files.
Chaincode	Analyze business rules, functions, validation logic, CouchDB indexes, and transaction handling.
PostgreSQL	Query off-chain tables such as wallets, transactions, organizations, audit logs, and dashboard metrics.
CouchDB	Inspect world state structure, indexes, and query model design.
Node.js / Express API	Review routes, controllers, validation, error handling, security middleware, and Fabric SDK integration.
Angular Test UI	Review screens, services, API calls, routing, forms, and dashboard integration.
Docker Compose	Check container status, service health, logs, networking, and environment problems.
Documentation	Generate handover documents, API docs, test plans, dashboard specs, and leadership reports.

3. Recommended MCP Servers

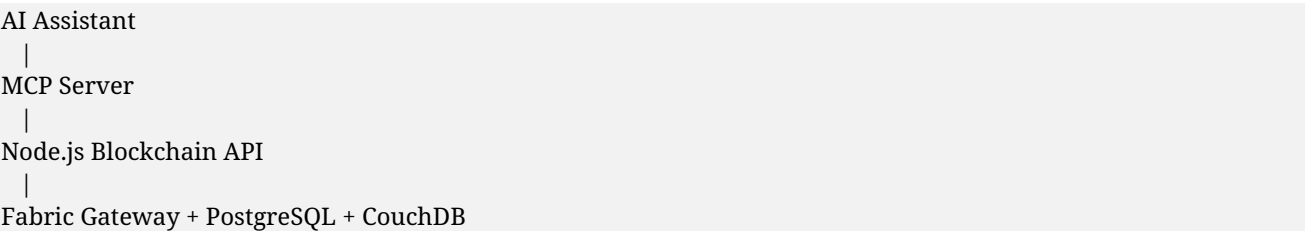
MCP Server	Recommended Usage in Your Project
Filesystem MCP	Read files from /home/nix/u01/blockchain-integration. Use it for chaincode, API, Angular UI, Docker compose files, and documentation.
PostgreSQL MCP	Query the blockchain schema safely. Use a read-

	only user first for wallets, organizations, transactions, and audit logs.
GitHub MCP	Inspect repository branches, issues, pull requests, commits, and code changes.
Docker MCP	Inspect running containers, unhealthy services, logs, and service restart issues.
Playwright / Browser MCP	Test the Angular blockchain-test-ui screens automatically from the browser.
Custom Fabric MCP	Expose safe blockchain actions such as status checks, wallet queries, balance queries, and transaction history queries.

4. Recommended Architecture



Best practice: use your existing Node.js Blockchain API as the safe gateway for blockchain-related MCP actions. Avoid giving MCP direct production-level Fabric CLI write access.



5. Setup Phases by Order

Phase	MCP Server	Goal
Phase 1	Filesystem MCP	Allow the AI assistant to read project files only inside /home/nix/u01/blockchain-integration.
Phase 2	PostgreSQL MCP	Allow safe read-only database analysis for wallets, transactions, organizations, audit logs, and

		dashboard metrics.
Phase 3	GitHub MCP	Allow repository inspection, issue tracking, pull request review, and code comparison.
Phase 4	Docker MCP	Allow service health checks, log review, and container debugging.
Phase 5	Playwright MCP	Allow automated Angular UI testing and console error checks.
Phase 6	Custom Fabric MCP	Expose safe blockchain status/query tools through your Node.js API.

6. Phase 1 - Filesystem MCP

Start with filesystem access because it allows the AI assistant to understand your project structure without touching the database or blockchain network.

Recommended Project Root

```
/home/nix/u01/blockchain-integration
```

Important Safety Rule

- Do not expose the full /home/nix folder.
- Do not expose private folders unrelated to the project.
- Start with read-only access if your MCP client supports it.

7. Phase 2 - PostgreSQL MCP

PostgreSQL MCP is very useful for your dashboard and reconciliation work. Start with a read-only database user.

Recommended Read-Only User

```
CREATE USER mcp_reader WITH PASSWORD 'StrongPasswordHere';

GRANT CONNECT ON DATABASE vfds_dev TO mcp_reader;
GRANT USAGE ON SCHEMA blockchain TO mcp_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA blockchain TO mcp_reader;

ALTER DEFAULT PRIVILEGES IN SCHEMA blockchain
GRANT SELECT ON TABLES TO mcp_reader;
```

Useful Dashboard Queries MCP Can Help With

Dashboard Metric	Database Source
Total Balance	SUM(current_balance) from blockchain.wallets
Today Created Wallets	COUNT wallets created today
Today Transactions	COUNT transactions created today
Total Transactions	COUNT all transactions
Wallet Growth Chart	wallet count grouped by date

Transactions Overview	transactions grouped by type/status/date
Organization / Bank Summary	wallets and balances grouped by organization
Failed Transactions	transactions where status = failed
Pending Transactions	transactions where status = pending

8. Phase 3 - Docker MCP

Docker MCP helps the AI assistant inspect your running blockchain environment and identify service-level issues.

Service	MCP Can Check
Fabric peer	Container status, logs, TLS/MSP errors, chaincode container errors.
Orderer	Availability, channel/orderer logs, consensus or communication errors.
CouchDB	Availability, database creation, index errors, peer-CouchDB connectivity.
PostgreSQL	Availability, connection failures, schema access problems.
Node.js API	Runtime errors, Fabric SDK errors, validation errors, API logs.
Angular UI	Container status, Nginx serving issues, frontend build/deployment problems.

9. Phase 4 - Playwright / Browser MCP

Browser MCP is useful for testing your Angular blockchain-test-ui like a QA engineer.

Screen	What to Test
Dashboard	Metric cards, charts, organization summary, blockchain network health.
Wallet Create	Form validation, API call, success state, duplicate customer handling.
Wallet Login	Customer-session logic, wallet profile storage, error messages.
Wallet Query	Wallet lookup, balance, organization, status, and empty state.
Wallet Transfer	Sender wallet prefill, amount validation, transaction submission.
Organization Transfer	Organization selection, amount validation, transaction submission.
Transaction History	Filters, pagination, empty state, error state, transaction detail display.
Fabric Test	Evaluate and submit test functions, error visibility,

response formatting.

10. Phase 5 - Custom Fabric MCP Server

A custom Fabric MCP server should expose controlled, safe operations only. For production-style safety, let the MCP server call your Node.js Blockchain API instead of running raw Fabric CLI commands.

MCP Tool Name	Purpose
get_blockchain_status	Return channel, peer, orderer, CouchDB, PostgreSQL, and chaincode status.
get_wallet	Query wallet by walletAddress or customerId.
get_wallet_balance	Return current wallet balance.
get_transaction_history	Return transaction history for a wallet.
get_organization_summary	Return organization-level wallet count, balance, and transaction counts.
get_dashboard_metrics	Return dashboard KPIs for the online dashboard.
create_test_wallet	Create a test wallet only in a controlled environment.
submit_test_transfer	Submit a controlled test transfer only in development.

11. MCP Usage for Your Blockchain Dashboard

Dashboard Feature	Recommended MCP Source
Total Balance	PostgreSQL MCP
Today Created Wallets	PostgreSQL MCP
Today Transactions	PostgreSQL MCP
Total Transactions	PostgreSQL MCP
Wallet Growth Chart	PostgreSQL MCP
Transactions Overview	PostgreSQL MCP
Wallet-to-Wallet Transfers	PostgreSQL MCP
Wallet-to-Organization Transfers	PostgreSQL MCP
Total Transaction Volume	PostgreSQL MCP
Average Transaction Amount	PostgreSQL MCP
Failed / Pending Transactions	PostgreSQL MCP
Organization / Bank Summary	PostgreSQL MCP
Fabric Peer Status	Docker MCP or Custom Fabric MCP
Orderer Status	Docker MCP or Custom Fabric MCP
CouchDB Status	Docker MCP
PostgreSQL Status	PostgreSQL MCP
Chaincode Status	Custom Fabric MCP
Channel Name	Custom Fabric MCP
Last Block Number	Custom Fabric MCP

12. Security Rules

Security Area	Recommended Rule
Database Access	Start with read-only database users. Do not give admin access to MCP unless absolutely required.
Filesystem Access	Limit access to the project folder only. Avoid full home-directory access.
Secrets	Do not expose .env files by default. Mask API keys, private keys, certificates, and database passwords.
Production Data	Use masked, anonymized, or test data when possible.
Write Actions	Disable write actions by default. Enable only specific test actions in development.
Fabric Transactions	Do not allow unrestricted submit transactions. Use controlled API endpoints only.
Logs	Avoid exposing customer private data in logs returned to the AI assistant.
API Keys	Use environment variables or secret stores. Do not paste keys directly into prompts.

13. Ready-to-Use MCP Prompts

A. Full Project Analysis Prompt

Act as a Senior Blockchain Integration Engineer.

Using MCP access to my project folder and PostgreSQL database, analyze my Blockchain Integration Project.

Project root:

/home/nix/u01/blockchain-integration

Review:

1. Hyperledger Fabric network files
2. Chaincode folder
3. Node.js blockchain API
4. Angular blockchain-test-ui
5. PostgreSQL blockchain schema
6. Docker compose files

Then provide:

- current architecture summary
- detected issues
- missing dashboard APIs
- missing database indexes
- missing Fabric health checks
- security risks
- recommended next steps by priority

B. Chaincode Review Prompt

Check my Hyperledger Fabric chaincode in:
/home/nix/u01/blockchain-integration/chaincode/kyc-wallet-chaincode-js

Review:

- CreateWallet
- LoginWallet
- TransferBetweenWallets
- TransferToOrganization
- GetWalletBalance
- GetTransactionHistory

Find business rule issues, security issues, missing validations, and CouchDB index requirements.

C. PostgreSQL Dashboard Prompt

Using PostgreSQL MCP, check the blockchain schema.

Show:

1. total wallets
2. total transactions
3. wallet-to-wallet transactions
4. wallet-to-organization transactions
5. failed transactions
6. pending transactions
7. total balance by organization
8. transaction volume by day

D. API Debugging Prompt

Review my Node.js blockchain API.

Check:

- /api/v1/wallets
- /api/v1/wallets/login
- /api/v1/transactions/wallet-transfer
- /api/v1/transactions/organization-transfer
- /api/v1/wallets/:walletAddress/balance
- /api/v1/transactions

Find route errors, validation problems, missing error handling, security issues, and PostgreSQL save issues.

E. Docker Debugging Prompt

Check my Docker blockchain environment.

Verify:

- Fabric peer containers
- orderer container
- CouchDB containers
- PostgreSQL container
- blockchain API container
- Angular UI container

Then explain any unhealthy container or log error.

F. Angular UI Testing Prompt

Use browser testing to check the Blockchain Test UI.

Test screens:

- Dashboard
- Wallet Create
- Wallet Login
- Wallet Query
- Wallet Transfer
- Organization Transfer
- Transaction History
- Fabric Test

Check layout, API calls, console errors, loading states, empty states, and validation messages.

14. Recommended First Implementation Plan

Step	Action	Expected Result
1	Install and configure Filesystem MCP for the project folder only.	AI can inspect code, configs, chaincode, and docs.
2	Create read-only PostgreSQL user mcp_reader.	AI can query dashboard and reconciliation data safely.
3	Connect PostgreSQL MCP using the read-only user.	AI can generate SQL checks and dashboard metrics.
4	Add GitHub MCP if the project is in a repository.	AI can review branches, commits, issues, and PRs.
5	Add Docker MCP for local/development environment only.	AI can inspect containers and logs.
6	Add Playwright MCP for Angular UI testing.	AI can test screens and report frontend issues.
7	Build custom Fabric MCP through Node.js API.	AI can safely query blockchain status and read-only blockchain data.

15. Final Recommendation

Start small and safe. Use Filesystem MCP and PostgreSQL MCP first. After that, add Docker MCP, Playwright MCP, and finally a custom Fabric MCP server through your existing Node.js Blockchain API.

Your first goal should be:

AI assistant can read the project files and query PostgreSQL dashboard data safely.

Your second goal should be:

AI assistant can inspect Docker logs and test the Angular UI.

Your final advanced goal should be:

AI assistant can call safe Fabric status and query tools through the Node.js Blockchain API.